
raft-rx — User guide: C

FSM-based Raft implementation over rxnet

2026-05-19

Contents

raft_rx - C User Guide	7
Part I — The Library	9
1. Overview	9
How to read this guide	9
2. Architecture	10
3. Key concepts	11
Roles	11
Log, terms, and commit	11
Membership configuration	11
4. Implementing your application	12
4.1 The application interface	12
4.2 Example: replicated counter	13
5. Setting up the cluster	14
5.1 Create the runtime and cluster	14
5.2 Configure the node	14
5.3 Add the node to the cluster	15
6. Choosing a transport	15
6.1 Memory transport (in-process — tests and simulations)	15
6.2 TCP transport (multi-process — production)	15
6.3 Custom transport	16
7. Running the event loop	17
7.1 Cooperative executor	17
7.2 Cyclic executive	18
7.3 Thread executor	18
8. Submitting commands	18
9. Membership changes	19
9.1 Adding a node	19
9.2 Removing a node	19
9.3 Joint consensus internals	20

10. TCP transport — persistence	20
Peer table (peers.txt)	20
Own address (self.txt)	20
11. Persistence and storage format	21
meta.txt	21
log.txt	22
snapshot.bin	22
12. Configuration reference	22
raft_node_config_t	22
Timing guidelines	23
Restart behaviour	23
13. Limits reference	23
Constant table	24
Memory footprint	24
ESP32 / embedded profile	25
14. Building	26
Library	26
Linking your application	26
Minimal example	26
15. Troubleshooting	26
No leader elected after startup	26
Compaction / snapshot	27
raft_cluster_add_node returns NULL	27
Part II — The Demo Application	29
16. Overview	29
What the demo app adds on top of raft_rx	29
Source layout	29
17. Building	30
18. Starting a fresh cluster	30
19. Restarting a node	31
20. Adding a node	31
21. Removing a node	31
22. CLI reference	32
Key-value commands	32
Cluster inspection	32
Cluster management	32
Fault injection	33

merge HOST:PORT — cluster merge protocol	33
23. Command-line options	34

List of Figures

raft_rx - C User Guide

Part I — The Library

1. Overview

`raft_rx` is a C library that implements the Raft consensus algorithm on top of the `rxnet` reactive-synchronous runtime. It provides:

- **Replicated state machine** — commands are appended to a distributed log, committed by majority quorum, and applied to your application in strict order.
- **Leader election** — automatic election with randomised timeouts; the cluster self-heals after a leader failure with no operator intervention.
- **Membership changes** — safe joint-consensus reconfiguration; nodes can be added or removed while the cluster is running.
- **Persistent storage** — term, log, snapshot, and membership configuration survive process restarts.
- **Pluggable transport** — swap the built-in in-memory transport for TCP (or any other medium) by implementing five function pointers.
- **Pluggable application** — implement four callbacks to connect your own state machine; the library is otherwise agnostic to what the commands mean.

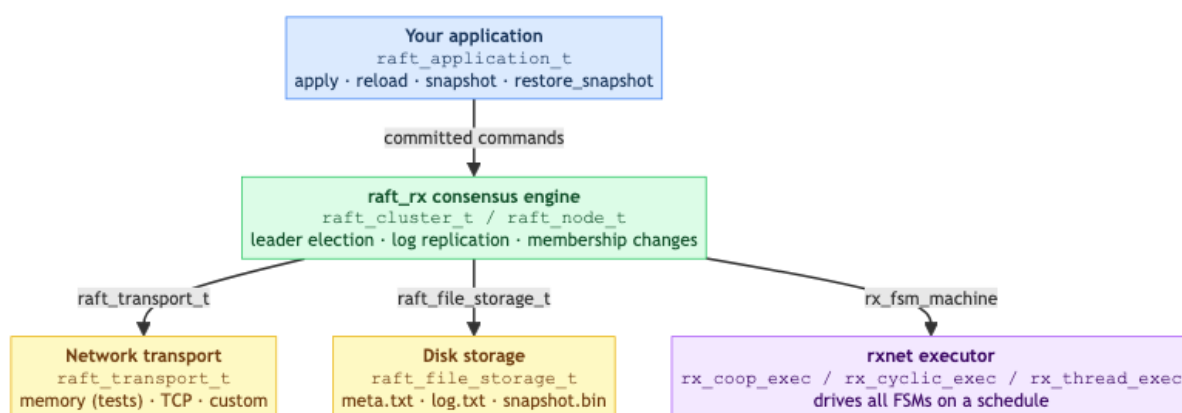
How to read this guide

Use Part I when you are integrating the library into your own C process. The most important path is:

1. Implement `raft_application_t`.
2. Create a `rx_fsm_runtime` and a `raft_cluster_t`.
3. Add one `raft_node_t` with persistent storage.
4. Attach either the memory transport, TCP transport, or your own `raft_transport_t`.
5. Run an rxnet executor.

Use Part II when you want a complete runnable example. The demo application is deliberately small: it shows the same integration points with a key-value state machine, TCP networking, a CLI, persistence and dynamic membership.

2. Architecture



Layer	Type	Responsibility
Application	<code>raft_application_t</code>	Your state machine — what commands mean
Consensus engine	<code>raft_cluster_t / raft_node_t</code>	Raft protocol: leader election, log replication, membership
Transport	<code>raft_transport_t</code>	Send/receive messages between nodes
Storage	<code>raft_file_storage_t</code>	Durable persistence of Raft state
Runtime	<code>rx_*_exec</code>	Cooperative/cyclic/thread scheduling of all FSMs

3. Key concepts

Roles

Every node is always in exactly one of three roles:

Role	Description
Follower	Passive; accepts log entries replicated by the leader
Candidate	Running for election after the election timeout expires
Leader	Accepts client commands, replicates them, commits when quorum reached

Log, terms, and commit

- Each client command becomes a **log entry** with fields (`index`, `term`, `command`).
- The leader replicates entries to a quorum of nodes, then marks them **committed**.
- Committed entries are applied to every node's application in index order via `apply`.
- A **term** is a monotonically increasing counter that advances with each election — the Raft equivalent of a logical clock.

Membership configuration

`raft_rx` tracks which nodes form the authoritative quorum as `raft_cluster_configuration_t`:

```

1 typedef struct {
2     char    old_members[RAFT_MAX_NODES][RAFT_MAX_ID]; /* current or
3     leaving set */
4     size_t  old_count;
5     char    new_members[RAFT_MAX_NODES][RAFT_MAX_ID]; /* joining set (
6     joint only) */
7     size_t  new_count;
8     int     index; /* log index where this config was written */
9 } raft_cluster_configuration_t;
```

`old_count > 0`, `new_count == 0` means **stable** configuration.

`old_count > 0`, `new_count > 0` means **joint** configuration (transition in progress).

The configuration is persisted in `meta.txt` and replayed on restart.

4. Implementing your application

4.1 The application interface

Connect your state machine to the consensus engine by filling in four callbacks:

```
1 typedef struct {
2     void (*apply)          (void *user, const raft_command_t *
        command);
3     void (*reload)         (void *user);
4     size_t (*snapshot)     (void *user, void *buf, size_t buf_size)
        ;
5     void (*restore_snapshot) (void *user, const void *buf, size_t
        size);
6     void *user;
7 } raft_application_t;
```

A command is a fixed-size triple of C strings that the library treats as opaque:

```
1 typedef struct {
2     char op    [32];          /* operation name — "set", "del", "inc
        ", ... */
3     char key   [RAFT_MAX_KEY]; /* primary key      (max 63 chars)
        */
4     char value[RAFT_MAX_VALUE]; /* payload / value (max 127 chars)
        */
5 } raft_command_t;
```

Callback	When it is called	What you must do
<code>apply</code>	Once per committed log entry, in index order	Mutate your state — this is the only place you should do so
<code>reload</code>	Restart with no snapshot on disk	Reset to the empty state; the engine re-applies the full log via <code>apply</code>
<code>snapshot</code>	Log compaction	Serialise your current state into <code>buf</code> (max <code>buf_size</code> bytes); return bytes written

Callback	When it is called	What you must do
<code>restore_snapshot</code>	Restart with a snapshot on disk	Deserialise <code>buf</code> into your state; the engine then re-applies any log entries that follow

Correctness rule: `apply` is called exactly once per entry, always on the rxnet compute thread. Do **not** mutate your state outside of `apply`, and do **not** spawn threads inside it.

Snapshot size: The snapshot buffer is `RAFT_MAX_SNAPSHOT_SIZE` bytes (default 4096). Override it before including `raft.h` if your state is larger:

```
1 #define RAFT_MAX_SNAPSHOT_SIZE (256 * 1024)
2 #include "raft/raft.h"
```

4.2 Example: replicated counter

```
1 typedef struct { int counter; } my_state_t;
2
3 static void my_apply(void *user, const raft_command_t *cmd) {
4     my_state_t *s = (my_state_t *)user;
5     if (strcmp(cmd->op, "inc") == 0)
6         s->counter += atoi(cmd->value);
7     else if (strcmp(cmd->op, "reset") == 0)
8         s->counter = 0;
9 }
10
11 static void my_reload(void *user) {
12     ((my_state_t *)user)->counter = 0; /* reset; log replay rebuilds
13     state */
14 }
15
16 static size_t my_snapshot(void *user, void *buf, size_t buf_size) {
17     int n = snprintf((char *)buf, buf_size, "%d\n",
18                     ((my_state_t *)user)->counter);
19     return (n > 0 && (size_t)n < buf_size) ? (size_t)n : 0;
20 }
21
22 static void my_restore_snapshot(void *user, const void *buf, size_t
23     size) {
24     (void)size;
25     ((my_state_t *)user)->counter = atoi((const char *)buf);
26 }
```

```
26 raft_application_t my_make_app(my_state_t *s) {
27     raft_application_t app;
28     app.apply          = my_apply;
29     app.reload         = my_reload;
30     app.snapshot       = my_snapshot;
31     app.restore_snapshot = my_restore_snapshot;
32     app.user           = s;
33     return app;
34 }
```

5. Setting up the cluster

5.1 Create the runtime and cluster

```
1 rx_fsm_runtime runtime;
2 raft_cluster_t cluster;
3
4 /* capacity = total number of rx_fsm_machines you will register
5  * (one per Raft node + one per auxiliary FSM such as a CLI or sensor
6  *   reader) */
7 rx_fsm_runtime_init(&runtime, 2);
8 raft_cluster_init(&cluster, &runtime);
9 raft_cluster_enable_realtime_clock(&cluster); /* use wall-clock time
10 */
```

5.2 Configure the node

```
1 raft_node_config_t cfg;
2 memset(&cfg, 0, sizeof(cfg));
3
4 strncpy(cfg.node_id, "n1", RAFT_MAX_ID - 1);
5 cfg.election_timeout_ms = 500; /* base timeout before starting
6   election */
7 cfg.heartbeat_interval_ms = 100; /* leader sends heartbeats at this
8   rate */
9
10 /* IDs of the other nodes this node will talk to.
11  * The transport uses these to route outgoing messages. */
12 strncpy(cfg.peers[0], "n2", RAFT_MAX_ID - 1);
13 strncpy(cfg.peers[1], "n3", RAFT_MAX_ID - 1);
14 cfg.peer_count = 2;
```

```

14 /* IDs of ALL nodes in the initial cluster — must be identical on every
    node.
15  * Leave empty (initial_member_count = 0) for nodes joining an existing
    cluster. */
16 strncpy(cfg.initial_members[0], "n1", RAFT_MAX_ID - 1);
17 strncpy(cfg.initial_members[1], "n2", RAFT_MAX_ID - 1);
18 strncpy(cfg.initial_members[2], "n3", RAFT_MAX_ID - 1);
19 cfg.initial_member_count = 3;

```

5.3 Add the node to the cluster

```

1 my_state_t state = {0};
2 raft_application_t app = my_make_app(&state);
3
4 /* root_dir: the engine stores node state under {root_dir}/{node_id}/
5  * Pass NULL to disable persistence (useful in tests). */
6 raft_node_t *node = raft_cluster_add_node(&cluster, &cfg,
7                                           "var/myapp", /* root_dir */
8                                           &app,
9                                           10000);      /* tick period
                                                         μs */

```

6. Choosing a transport

6.1 Memory transport (in-process — tests and simulations)

All nodes share the same `raft_cluster_t`. No threads, no network.

```
1 node->transport = raft_mem_transport_make(node);
```

Use `raft_cluster_tick(cluster, dt_ms)` to drive time manually, which is convenient for deterministic tests:

```

1 for (int i = 0; i < 100; ++i)
2     raft_cluster_tick(&cluster, 10); /* simulate 1000 ms */

```

6.2 TCP transport (multi-process — production)

Each process hosts one node. Include `raft/raft_tcp_transport.h` and link `libraft_rx.a`.


```

1 raft_tcp_transport_t tcp;
2
3 raft_tcp_transport_init(&tcp, 5001);           /* 1. listen
   port */
4 raft_tcp_transport_set_self(&tcp, "n1", "127.0.0.1"); /* 2. own
   identity */
5 raft_tcp_transport_set_data_dir(&tcp, "var/myapp/n1"); /* 3. load
   peers.txt / */
6
                                   /* self.txt
                                   from disk */
7 /* Only needed on first run – persisted automatically afterwards */
8 raft_tcp_transport_add_peer(&tcp, "n2", "127.0.0.1", 5002);
9 raft_tcp_transport_add_peer(&tcp, "n3", "127.0.0.1", 5003);
10
11 raft_tcp_transport_start(&tcp); /* 4. spawn listener thread */
12
13 node->transport = raft_tcp_transport_make(&tcp); /* 5. attach to node
   */

```

Call order matters: `set_self` and `set_data_dir` must precede `start`.

Stop the transport after the event loop exits:

```

1 raft_tcp_transport_stop(&tcp);

```

6.3 Custom transport

Implement the five-function vtable and assign it to `node->transport`:

```

1 typedef struct {
2     int (*send) (void *ctx, const raft_message_t *msg);
3     size_t (*recv) (void *ctx, raft_message_t *out, size_t
   capacity);
4     int (*submit_command) (void *ctx, const raft_command_t *cmd);
5     size_t (*recv_commands) (void *ctx, raft_command_t *out, size_t
   capacity);
6     void (*set_enabled) (void *ctx, int enabled); /* may be NULL
   */
7     void *ctx;
8 } raft_transport_t;

```

Function	Called by	Contract
<code>send</code>	Engine	Deliver <code>msg</code> to the node identified by <code>msg->target</code> ; return 0 on success

Function	Called by	Contract
<code>recv</code>	Engine	Copy at most <code>capacity</code> incoming messages into <code>out</code> ; return count
<code>submit_command</code>	Your client code	Enqueue <code>cmd</code> for the leader to process; return 0 on success
<code>recv_commands</code>	Engine (leader only)	Drain at most <code>capacity</code> pending commands into <code>out</code> ; return count
<code>set_enabled</code>	<code>raft_node_stop</code> / <code>raft_node_start</code>	Gate the transport; pass NULL if you do not need simulated failures

`send` / `recv` carry `raft_message_t` (Raft protocol messages).

`submit_command` / `recv_commands` carry `raft_command_t` (application commands).

7. Running the event loop

Raft nodes are `rx_fsm_machine` instances driven by an rxnet executor. Pick the executor that fits your application:

7.1 Cooperative executor

Single-threaded, polls continuously, sleeps between ticks. Simplest option.

```

1 #include "rxnet/coop.h"
2
3 rx_coop_exec ce;
4 rx_coop_exec_init(&ce);
5 rx_coop_exec_add(&ce, &runtime.runtime);
6 rx_coop_exec_run(&ce); /* blocks until stopped */

```

7.2 Cyclic executive

Fixed-period scheduling driven by a sleep-until timer. Drop-in replacement for `rx_coop_exec`; the period is read from each machine's `period_us`.

```
1 #include "rxnet/cyclic.h"
2
3 rx_cyclic_exec ce;
4 rx_cyclic_exec_init(&ce);
5 rx_cyclic_exec_add(&ce, &runtime.runtime);
6 rx_cyclic_exec_run(&ce);
```

7.3 Thread executor

One pthread per FSM node, synchronised with BSP barriers (latch / evaluate / commit phases run in lock-step across all threads). Suitable when the Raft FSM and other FSMs should run in parallel.

```
1 #include "rxnet/thread.h"
2
3 rx_thread_exec te;
4 rx_thread_exec_init(&te);
5 rx_thread_exec_add(&te, &runtime.runtime);
6 rx_thread_exec_run(&te); /* last node of last runtime stays on main
   thread */
```

With the TCP transport all shared mutable state between the listener thread and the compute thread is already protected by a mutex, so the thread executor is safe to use.

8. Submitting commands

Commands must reach the **leader** to be committed.

Memory transport — find the leader directly and submit:

```
1 raft_node_t *leader = raft_cluster_leader(&cluster);
2 if (leader) {
3     raft_command_t cmd;
4     memset(&cmd, 0, sizeof(cmd));
5     strncpy(cmd.op, "inc", sizeof(cmd.op) - 1);
6     strncpy(cmd.value, "1", sizeof(cmd.value) - 1);
7     raft_node_submit_command(leader, &cmd);
8 }
```

TCP transport — enqueue into the transport; the leader drains it on the next tick:

```
1 raft_command_t cmd;
2 memset(&cmd, 0, sizeof(cmd));
3 strncpy(cmd.op, "inc", sizeof(cmd.op) - 1);
4 strncpy(cmd.value, "1", sizeof(cmd.value) - 1);
5 node->transport.submit_command(node->transport.ctx, &cmd);
```

Commands submitted to a follower are silently dropped; the application is responsible for directing commands to the leader (or forwarding them).

9. Membership changes

9.1 Adding a node

A joining node sets `cfg.learner = 1` (prevents it from auto-bootstrapping a solo cluster) and leaves `initial_member_count = 0`. With the TCP transport, it calls `raft_tcp_transport_request_join` once at startup to contact any one existing member (the *introducer*):

```
1 cfg.learner = 1;
2 /* no initial_members */
3
4 /* After attaching the transport: */
5 raft_tcp_transport_request_join(&tcp,
6     "n4", "127.0.0.1", 5004, /* this node's identity */
7     "127.0.0.1", 5001);      /* introducer address */
```

The join protocol runs automatically:

```
1 n4 — (join-request) —> n1
2 n1 — (flood: forwarded join-request) —> n2, n3
3 n1, n2, n3 — (announce-self) —> n4
4 leader appends enter_joint / leave_joint to log
5 n4 is a full voting member once leave_joint commits
```

Existing nodes do **not** need reconfiguration.

9.2 Removing a node

Call `raft_node_request_membership_change` on the leader, passing the **target full membership list** (not a delta):

```

1 /* Remove n4: new membership is {n1, n2, n3} */
2 char new_members[3][RAFT_MAX_ID] = {"n1", "n2", "n3"};
3 raft_node_request_membership_change(leader_node, new_members, 3);

```

To remove the current node itself, exclude its own ID from the list. The engine runs joint consensus automatically; the removed node stops once the final configuration commits.

9.3 Joint consensus internals

Membership changes use the two-phase joint consensus approach from the Raft paper.

Phase 1 — enter joint: the leader appends a `cluster.enter_joint` log entry encoding both old and new member sets. While this entry is not yet committed, quorum requires a majority from **both** sets.

Phase 2 — leave joint: once all new members have caught up and `enter_joint` is committed, the leader appends `cluster.leave_joint`. The cluster transitions to the new stable configuration.

```

1 log: ... | enter_joint [old={n1,n2,n3}, new={n1,n2,n3,n4}] | ... |
      leave_joint [new={n1,n2,n3,n4}] | ...

```

10. TCP transport — persistence

Peer table (`peers.txt`)

Every time a peer is registered — via `raft_tcp_transport_add_peer` or through the join protocol — the full peer table is written atomically to `{data_dir}/peers.txt`:

```

1 n2 127.0.0.1 5002
2 n3 127.0.0.1 5003

```

`set_data_dir` reloads this file at startup, so peer addresses do not need to be supplied again after the first run. `add_peer` deduplicates by node ID: adding the same ID with a new address updates the entry and rewrites the file.

Own address (`self.txt`)

`raft_tcp_transport_start` writes `{data_dir}/self.txt`:

```
1 127.0.0.1 5001
```

`set_data_dir` reads it at startup and fills in `own_host` and `listen_port` if they are not already set by the caller. Priority (highest wins):

1. Explicit value passed by the caller (`init` port, `set_self` host)
2. Values read from `self.txt`
3. Defaults (127.0.0.1, port 0 — no listening)

All writes use an atomic write-to-temp / rename pattern so a crash mid-write never corrupts the live file.

11. Persistence and storage format

Each node stores its state under `{root_dir}/{node_id}/`:

1	<code>var/myapp/</code>	└─
2	<code>n1/</code>	└─
3	<code>meta.txt</code>	Raft durable state (term, voted_for, config, ...) └─
4	<code>log.txt</code>	Log entries not yet compacted into a snapshot └─
5	<code>snapshot.bin</code>	Compacted application state └─
6	<code>peers.txt</code>	Known TCP peer addresses (TCP transport only) └─
7	<code>self.txt</code>	Own host and listen port (TCP transport only)

meta.txt

Plain text, one `key value` pair per line:

```
1 current_term 7
2 voted_for n2
3 compaction_threshold 128
4 commit_index 14
5 snapshot_last_included_index 10
6 snapshot_last_included_term 4
7 configuration_index 3
8 old_count 3
9 old_member n1
10 old_member n2
11 old_member n3
12 new_count 0
```

log.txt

One entry per line, pipe-delimited `index | term | op | key | value`:

```

1 11|4|set|foo|bar
2 12|4|del|old|-
3 13|7|cluster.enter_joint|members|n1,n2,n3,n4
4 14|7|cluster.leave_joint|members|n1,n2,n3,n4

```

snapshot.bin

Binary: `[int32 index][int32 term][int32 size][<size> bytes of application data]`

The application data is whatever your `snapshot` callback writes. The engine validates `index` and `term` on load before passing the bytes to `restore_snapshot`.

12. Configuration reference

raft_node_config_t

Field	Type	Description
<code>node_id</code>	<code>char[RAFT_MAX_ID]</code>	Unique node name (max 15 chars)
<code>peers / peer_count</code>	<code>char[][RAFT_MAX_ID] / size_t</code>	IDs of the other nodes
<code>initial_members / initial_member_count</code>	<code>char[][RAFT_MAX_ID] / size_t</code>	IDs of all nodes in the initial cluster
<code>learner</code>	<code>int</code>	1 for a joining node — skips auto-bootstrap
<code>election_timeout_ms</code>	<code>int</code>	Base follower election timeout
<code>heartbeat_interval_ms</code>	<code>int</code>	Leader heartbeat interval

Timing guidelines

Parameter	Recommended	Minimum	Notes
<code>election_timeout_ms</code>	500 ms	$3-5 \times \text{RTT}$	Too small: spurious elections; too large: slow failover
<code>heartbeat_interval_ms</code>	100 ms	$\sim \text{RTT}$	Rule of thumb: at most 1/5 of <code>election_timeout_ms</code>
<code>tick_period_us</code>	10 000 μs	—	Keep at most $\frac{\text{heartbeat_interval_ms} \times 1000}{2}$

Restart behaviour

A node that has `current_term > 0` on disk (i.e. has participated in at least one election) gets an extra $2 \times \text{election_timeout_ms}$ added to its first election deadline. This window gives the existing leader time to send heartbeats before the restarting node launches a new election.

13. Limits reference

All size constants use `#ifndef` guards and can be overridden at compile time. The recommended method is via compiler `-D` flags so the value is consistent across every translation unit without touching source files:

```
1 make CPPFLAGS="-DRAFT_MAX_NODES=3 -DRAFT_MAX_QUEUE=8 -DRAFT_MAX_BATCH=4"
```

or in a `Makefile`:

```
1 CPPFLAGS += -DRAFT_MAX_NODES=3 -DRAFT_MAX_QUEUE=8 -DRAFT_MAX_BATCH=4 \
2           -DRAFT_MAX_LOG=32 -DRAFT_MAX_KEY=32 -DRAFT_MAX_VALUE=64
```


Constant table

Constant	Default	Meaning
<code>RAFT_MAX_NODES</code>	8	Maximum nodes per cluster
<code>RAFT_MAX_PEERS</code>	<code>RAFT_MAX_NODES - 1</code>	Derived — set <code>RAFT_MAX_NODES</code> to change both
<code>RAFT_MAX_ID</code>	16	Node ID buffer size including <code>\0</code>
<code>RAFT_MAX_LOG</code>	128	Log entries kept before compaction is forced
<code>RAFT_MAX_QUEUE</code>	256	Message queue depth (outbox and pending queues per node)
<code>RAFT_MAX_BATCH</code>	16	Max log entries per <code>AppendEntries</code> RPC
<code>RAFT_MAX_KEY</code>	64	<code>raft_command_t.key</code> buffer size including <code>\0</code>
<code>RAFT_MAX_VALUE</code>	128	<code>raft_command_t.value</code> buffer size including <code>\0</code>
<code>RAFT_MAX_SNAPSHOT_SIZE</code>	4096	Snapshot buffer in bytes
<code>RAFT_KV_MAX_PAIRS</code>	128	Pairs in the built-in KV application

Memory footprint

`RAFT_MAX_QUEUE` and `RAFT_MAX_BATCH` are the dominant drivers of memory use because every queued message (`raft_message_t`) carries a full log-entry batch inline. The table below shows the effect on key struct sizes:

Build profile	<code>raft_message_t</code>	<code>raft_node_t</code>	<code>raft_cluster_t</code>
Desktop defaults (nodes=8, queue=256, batch=16, key=64, value=128)	4 064 B	4 199 KB	42 MB
ESP32 (nodes=3, queue=8, batch=4, log=32, key=32, value=64)	704 B	28 KB	104 KB

The 42 MB desktop default is intentional for server/simulation use; it allows `RAFT_MAX_NODES` × `RAFT_MAX_QUEUE` = 2 048 messages to be buffered simultaneously without dynamic allocation. On embedded targets the constants must be reduced.

ESP32 / embedded profile

For an ESP32 cluster with the MQTT transport, add these flags to your `CMakeLists.txt` or `idf_component.cmake`:

```

1 target_compile_definitions(${COMPONENT_LIB} PUBLIC
2     RAFT_MAX_NODES=3
3     RAFT_MAX_QUEUE=8
4     RAFT_MAX_BATCH=4
5     RAFT_MAX_LOG=32
6     RAFT_MAX_KEY=32
7     RAFT_MAX_VALUE=64
8     RAFT_MAX_SNAPSHOT_SIZE=512
9 )

```

This brings `raft_cluster_t` to ~104 KB, well within the 520 KB SRAM of an ESP32 (or ESP32-S3 PSRAM budget). The in-memory transport is not used with MQTT, so `raft_memory_transport_t` — the second-largest contributor — remains allocated but is effectively dead weight; a future refactor may make it optional.

Rule: `RAFT_MAX_PEERS` is always `RAFT_MAX_NODES` - 1 and is computed automatically. Never define `RAFT_MAX_PEERS` directly.

14. Building

Library

```
1 make -C c build/libraft_rx.a      # standard build
2 make -C c build/libraft_rx_trace.a # with rxnet tracing (-
    DRX_TRACE_ENABLE)
```

Linking your application

```
1 INCLUDES := -Ic/include -Irxnet/c/include
2 LIBS      := c/build/libraft_rx.a rxnet/c/build/librxnet.a -lpthread
3
4 $(TARGET): $(SRC) $(LIBS)
5     $(CC) -std=c99 -O2 $(INCLUDES) -o $@ $(SRC) $(LIBS)
```

`libraft_rx.a` contains the public C implementation: `raft.c`, `raft_transport.c`, `raft_storage.c`, `raft_tcp_transport.c`, and `raft_kv_app.c`.

Minimal example

```
1 cc -std=c99 -O2 \
2   -Ic/include -Irxnet/c/include \
3   my_app.c \
4   c/build/libraft_rx.a rxnet/c/build/librxnet.a \
5   -o my_app
```

15. Troubleshooting

No leader elected after startup

- Verify that all nodes declare the **same** `initial_members` list.
- Check that peer addresses are reachable and ports are open.
- Ensure `election_timeout_ms` is at least $3-5 \times$ the round-trip latency.

Compaction / snapshot

Compaction triggers automatically when the log reaches `compaction_threshold` entries (default: `RAFT_MAX_LOG` = 128). The engine calls `snapshot`, writes `snapshot.bin`, and truncates `log.txt`. Verify that your `snapshot` callback serialises **all** state, otherwise a restart after compaction will miss entries that were already compacted.

`raft_cluster_add_node` returns NULL

- The cluster is already at `RAFT_MAX_NODES` capacity.
 - The data directory cannot be created (check permissions).
-

Part II — The Demo Application

16. Overview

`c/examples/demo_app` is a **worked example** that shows how to build a multi-process Raft cluster using `raft_rx`. It implements a replicated key-value store with a readline CLI.

It is **one possible application** of the library, not the library itself. Read Part I to understand the underlying API; read this part to see it in action and to use the `raft_node` binary for experimentation.

What the demo app adds on top of `raft_rx`

Concern	How the demo app handles it
State machine	Built-in key-value store (<code>raft_kv_app</code>)
Transport	TCP (<code>raft_tcp_transport</code>)
Executor	<code>rx_coop_exec</code> (cooperative, single thread)
Command routing	Follower forwards commands to leader via TCP
CLI	<code>rx_fsm_machine</code> on the same runtime as the Raft node
Argument parsing	Custom <code>--id</code> , <code>--port</code> , <code>--member</code> , <code>--peer</code> , <code>--join</code> flags

Source layout

1	<code>c/examples/demo_app/</code>	
2	<code>Makefile</code>	

3	README.md	└─
4	src/	└─
5	main.c	process entry point and Raft/TCP/runtime composition
6	demo_args.*	argument parsing and validation
7	demo_kv_app.*	replicated KV application adapter
8	cli_fsm.*	interactive CLI FSM

17. Building

```
1 make -C c/examples/demo_app
2 # Result: c/examples/demo_app/build/raft_node
```

18. Starting a fresh cluster

All nodes in the initial cluster must agree on the same member list. Provide `--member ID` once per node and `--peer ID=HOST:PORT` for every other node.

```
1 # Terminal 1
2 ./build/raft_node --id n1 --port 5001 \
3   --member n1 --member n2 --member n3 \
4   --peer n2=127.0.0.1:5002 --peer n3=127.0.0.1:5003
5
6 # Terminal 2
7 ./build/raft_node --id n2 --port 5002 \
8   --member n1 --member n2 --member n3 \
9   --peer n1=127.0.0.1:5001 --peer n3=127.0.0.1:5003
10
11 # Terminal 3
12 ./build/raft_node --id n3 --port 5003 \
13   --member n1 --member n2 --member n3 \
14   --peer n1=127.0.0.1:5001 --peer n2=127.0.0.1:5002
```

After ~1 s a leader is elected.

A useful first exercise is:

```
1 n1> status
2 n1> leader
```

```
3 n1> set color blue
4 n2> get color
5 n3> log
```

`set` is a replicated write: it must be committed through the Raft log before it is applied. `get` is a local read from the node where you type it; it shows the state already applied on that process.

19. Restarting a node

All state (term, log, snapshot, peers, port, host) is persisted under `var/raft/{id}/` after the first run. Restart with only `--id`:

```
1 ./build/raft_node --id n1
```

The cluster does **not** need to be stopped. A 3-node cluster tolerates one offline node and stays fully operational while it restarts.

20. Adding a node

The joining node contacts any one existing member (`--join HOST:PORT`). It does **not** need `--member` or `--peer`:

```
1 ./build/raft_node --id n4 --port 5004 --join 127.0.0.1:5001
```

The introducer floods the request to all peers, each peer announces itself to the new node, and the leader applies the membership change via Raft. Existing nodes require no reconfiguration.

21. Removing a node

Use `rmnode` on whichever node is the current leader, or `leave` on the node that wants to exit:


```

1 n1> rmnode n4      # remove n4 (any node can issue this; only the leader
                      acts)
2 n4> leave          # n4 removes itself

```

22. CLI reference

Key-value commands

Command	Description
<code>set KEY VALUE</code>	Replicated write — forwarded to the leader automatically if this node is a follower
<code>get KEY</code>	Read from the local KV state already applied on this process
<code>delete KEY</code>	Replicated delete — forwarded to the leader if needed

Cluster inspection

Command	Description
<code>status</code>	Role, term, leader ID, commit index, log size, per-peer match index (leader only)
<code>leader</code>	Current leader node ID
<code>members</code>	Membership configuration — <code>stable</code> or <code>joint [old] [new]</code>
<code>log</code>	Full Raft log with <code>COMMITTED</code> / <code>UNCOMMITTED</code> markers per entry

Cluster management

Command	Who can run it	Description
<code>addnode ID</code>	Any node	Add <code>ID</code> to the cluster; forwarded to the leader automatically
<code>rmnode ID</code>	Any node	Remove <code>ID</code> from the cluster; forwarded to the leader automatically
<code>leave</code>	Any node	This node removes itself; forwarded to the leader if needed, then exits
<code>join HOST:PORT</code>	Any node	If currently a member, leaves first; then joins the cluster via the node at <code>HOST:PORT</code> as introducer (equivalent to <code>--join</code> at startup, but usable at runtime)
<code>merge HOST:PORT</code>	Any node	Cluster merge — see below
<code>port PORT</code>	Any node	Start listening on <code>PORT</code> (useful when the node was started without <code>--port</code>)

Fault injection

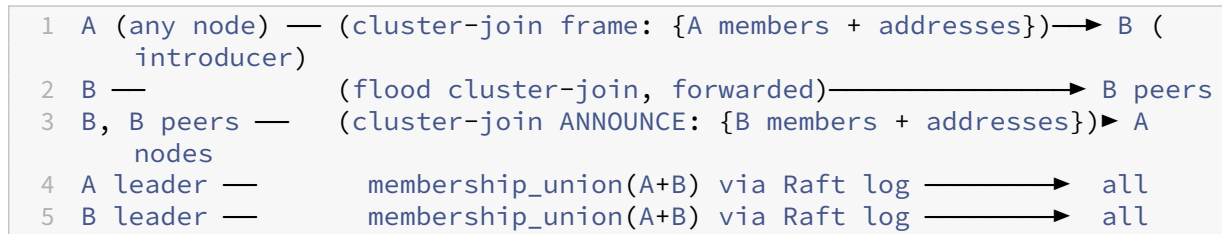
Command	Description
<code>stop</code>	Halts Raft processing on this node (simulates a crash without killing the process)
<code>start</code>	Resumes after <code>stop</code>

`merge HOST:PORT` — cluster merge protocol

`merge` connects **two independent clusters** into one. It is fundamentally different from `join`, which adds a single new node:

- **join** — one node introduces itself to an existing cluster.
- **merge** — this cluster introduces all of its members (with their TCP addresses) to the node at `HOST:PORT`, which belongs to a different cluster.

Protocol (initiated by running **merge** `HOST:PORT` on any node of cluster A):



1. The sender collects its full member list with TCP addresses and sends a `cluster_join` frame to `HOST:PORT`.
2. The receiving node floods the frame to all its own peers, and responds with its own full member list.
3. Both sides learn each other's TCP peers.
4. Each cluster's leader computes the **union** of both member sets (`A + B`) and initiates a joint-consensus membership change.
5. The result is a single merged cluster containing all nodes from both.

Use **merge** to join two previously independent clusters, or to reconnect a partitioned cluster after a network split is healed.

23. Command-line options

Option	First start	Restart	Description
<code>--id NAME</code>	Required	Required	Node identifier (max 15 chars)
<code>--port PORT</code>	Required	Optional	TCP listen port; persisted in <code>self.txt</code>
<code>--host HOST</code>	Optional	Optional	Advertised IP (default <code>127.0.0.1</code>)

Option	First start	Restart	Description
<code>--data DIR</code>	Optional	Optional	Data root directory (default <code>var/raft</code>)
<code>--member ID</code>	Required	Omit	Initial cluster member (repeat N times)
<code>--peer ID=HOST:PORT</code>	Required	Omit	Peer address (repeat N-1 times); persisted
<code>--join HOST:PORT</code>	To join	—	Join an existing cluster via this introducer